

**VIRTUAL ADDRESS :**

```
#include<stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

**OUTPUT :**

```
prane_partheev1@PARTHEEV:~$ nano VirtualAddress.c
prane_partheev1@PARTHEEV:~$ gcc VirtualAddress.c -o virtualAddress
prane_partheev1@PARTHEEV:~$ ./VirtualAddress
Value of x = 10
Virtual address of x = 0x7ffc3d9ce384
```

**DYNAMIC PROGRAMMING :**

```

#include<stdio.h>
#include<stdlib.h>

int global = 10;
int bss_variable;

int main() {
    int local = 20;
    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address      : %p\n", (void *)main);
    printf("Data address       : %p\n", (void *)&global);
    printf("BSS address        : %p\n", (void *)&bss_variable);
    printf("Heap address       : %p\n", (void *)heap);
    printf("Stack address      : %p\n", (void *)&local);

    free(heap);

    return 0;
}

```

OUTPUT :

```

prane_partheev1@PARTHEEV:~$ ls DynamicMemory.c
prane_partheev1@PARTHEEV:~$ gcc DynamicMemory.c -o DynamicMemory
prane_partheev1@PARTHEEV:~$ ./DynamicMemory
Code address      : 0x5a79c5c2e1a9
Data address       : 0x5a79c5c31010
BSS address        : 0x5a79c5c31018
Heap address       : 0x5a79f0a70010
Stack address      : 0x7fff4f5c40ac

```

DEMAND PAGING :

```

#include<stdio.h>
int main() {

    int page_table[4] = {1, -1, 3, -1};
    int page;

    printf("Enter page number (0-3): ");
    scanf("%d",&page);

    if(page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }
    return 0;
}

```

**OUTPUT :**

```

prane_partheev1@PARTHEEV:~$ nano RemandPaging.c
prane_partheev1@PARTHEEV:~$ gcc RemandPaging.c -o RemandPaging
prane_partheev1@PARTHEEV:~$ ./RemandPaging
Enter page number (0-3): 2
Page is already in memory.
Physical frame = 3

```

**ADDRESS :**

```

#include<stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address   = %d\n", virtual_address);
    printf("Page Number       = %d\n", page);
    printf("Offset               = %d\n", offset);
    printf("Frame Number          = %d\n", frame);
    printf("Physical Address = %d \n", physical_address);

    return 0;
}

```

OUTPUT :

```

prane_partheev1@PARTHEEV:~$ nano Address.c
prane_partheev1@PARTHEEV:~$ gcc Address.c -o Address
prane_partheev1@PARTHEEV:~$ ./Address
Virtual Address   = 250
Page Number       = 2
Offset           = 50
Frame Number      = 1
Physical Address = 150

```

MEMORY ALLOCATE :

```

#include<stdio.h>
#include<stdlib.h>

int main() {

    int *p;
    p = malloc(5 * sizeof(int));

    if(p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }
    for(int i=0;i<5;i++) {
        p[i] = (i + 1) * 10;
    }
    for(int i=0;i<5;i++) {
        printf("%d ",p[i]);
    }
    printf("\n");

    free(p);

    return 0;
}

```

OUTPUT :

```

prane_partheev1@PARTHEEV:~$ nano MemoryAllocate.c
prane_partheev1@PARTHEEV:~$ gcc MemoryAllocate.c -o MemoryAllocate
prane_partheev1@PARTHEEV:~$ ./MemoryAllocate
10 20 30 40 50

```

SEGMENTATION FAULT :

```
#include<stdio.h>

int main() {

    int *p = NULL;

    printf("Before accessong memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

OUTPUT :

```
prane_partheev1@PARTHEEV:~$ nano SegmentationFault.c
prane_partheev1@PARTHEEV:~$ gcc SegmentationFault.c -o SegmentationFault
prane_partheev1@PARTHEEV:~$ ./SegmentationFault
Before accessong memory
Segmentation fault ./SegmentationFault
```